Department of Computer Science and Engineering
University of Moratuwa

CS4533 – Concurrent Programming

Lab 05: Concurrent Data Structures

Module:	CS4533 – Concurrent Programming
Lab:	Lab 05 – Concurrent Data Structures
Duration:	2 hours (in-class)
Topics:	Coarse/fine-grained locking, CAS, lock-free counter, bounded queue

Prerequisites

- Completed Labs 01–04 (threads, mutexes, condition variables, semaphores, OpenMP)
- Familiarity with linked list and queue data structures in C
- Compile all exercises with: gcc -pthread -O2 -o output source.c
- Note: We use -O2 for this lab (not -O0) because performance comparison is the focus

Learning Objectives

By the end of this lab, you should be able to:

- Implement coarse-grained and fine-grained locking on a shared data structure
- Use Compare-and-Swap (CAS) to build lock-free algorithms
- Implement a concurrent bounded queue with split locks
- Reason about progress conditions: blocking, lock-free, wait-free

CAS Quick Reference

GCC provides built-in atomic operations. The key one for this lab is Compare-and-Swap (CAS):

```
// Returns true if *ptr was old_val and sets it to new_val
__sync_bool_compare_and_swap(ptr, old_val, new_val)

// Atomic fetch-and-add: adds val to *ptr, returns old value
__sync_fetch_and_add(ptr, val)

// Atomic fetch-and-subtract
__sync_fetch_and_sub(ptr, val)
```

Exercise 1: Coarse-Grained vs Fine-Grained Locking

Background

The simplest way to make a data structure thread-safe is to wrap every operation in a single global lock (coarse-grained). Fine-grained locking uses per-node locks, allowing operations on different parts of the list to proceed concurrently using hand-over-hand locking.

Provided Code

The following complete program includes the serial linked list, the benchmark harness, and main(). You only need to implement the functions marked TODO.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <limits.h>

typedef struct node {
    int key;
    struct node *next;
    pthread_mutex_t lock;
} node_t;

typedef struct {
    node_t *head;
    pthread_mutex_t global_lock;  // for coarse-grained
} list_t;

list_t *list_create() {
    list_t *l = malloc(sizeof(list_t));
    l->head = malloc(sizeof(node_t));
    l->head->key = INT_MIN;
    l->head->next = NULL;
    pthread_mutex_init(&l->head->lock, NULL);
    pthread_mutex_init(&l->global_lock, NULL);
    return l;
}

// --- Serial versions (reference) ---
int serial_insert(list_t *l, int key) {
    node_t *prev = l->head, *curr = prev->next;
    while (curr && curr->key < key)
        { prev = curr; curr = curr->next; }
    if (curr && curr->key == key) return 0;
    node_t *n = malloc(sizeof(node_t));
    n->key = key; n->next = curr;
    pthread_mutex_init(&n->lock, NULL);
    prev->next = n;
    return 1;
}
int serial_contains(list_t *l, int key) {
    node_t *c = l->head->next;
```

```
    while (c && c->key < key) c = c->next;
    return (c && c->key == key);
}
int serial_delete(list_t *l, int key) {
    node_t *prev = l->head, *curr = prev->next;
    while (curr && curr->key < key)
        { prev = curr; curr = curr->next; }
    if (!curr || curr->key != key) return 0;
    prev->next = curr->next;
    free(curr);
    return 1;
}

// ========================================
// TODO: Implement these two functions
// ========================================

// Coarse-grained: lock l->global_lock around
// the entire operation (same logic as serial)
int coarse_insert(list_t *l, int key) {
    // TODO
    return 0;
}
int coarse_contains(list_t *l, int key) {
    // TODO
    return 0;
}
int coarse_delete(list_t *l, int key) {
    // TODO
    return 0;
}

// Fine-grained: hand-over-hand locking.
// Lock prev, lock curr, unlock prev, advance.
// Always hold at least one lock while traversing.
int fine_insert(list_t *l, int key) {
    // TODO
    return 0;
}
int fine_contains(list_t *l, int key) {
    // TODO
    return 0;
}
int fine_delete(list_t *l, int key) {
    // TODO
    return 0;
}
```

Benchmark Harness (provided)

The following benchmark creates threads that perform a random mix of operations. It is already complete — do not modify it.

```
#define OPS_PER_THREAD 25000
```

```
#define KEY_RANGE 1000

typedef struct {
    list_t *list;
    int (*insert_fn)(list_t *, int);
    int (*contains_fn)(list_t *, int);
    int (*delete_fn)(list_t *, int);
    unsigned int seed;
} thread_arg_t;

void *worker(void *arg) {
    thread_arg_t *ta = (thread_arg_t *)arg;
    for (int i = 0; i < OPS_PER_THREAD; i++) {
        int key = rand_r(&ta->seed) % KEY_RANGE;
        int op = rand_r(&ta->seed) % 100;
        if (op < 50)
            ta->contains_fn(ta->list, key);
        else if (op < 75)
            ta->insert_fn(ta->list, key);
        else
            ta->delete_fn(ta->list, key);
    }
    return NULL;
}

double run_benchmark(list_t *list,
    int (*ins)(list_t*,int),
    int (*con)(list_t*,int),
    int (*del)(list_t*,int),
    int nthreads) {
    pthread_t threads[nthreads];
    thread_arg_t args[nthreads];
    struct timespec start, end;

    // Pre-populate list with 500 items
    for (int i = 0; i < 500; i++)
        serial_insert(list, rand() % KEY_RANGE);

    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < nthreads; i++) {
        args[i] = (thread_arg_t){list, ins, con,
                                 del, 42 + i};
        pthread_create(&threads[i], NULL,
                       worker, &args[i]);
    }
    for (int i = 0; i < nthreads; i++)
        pthread_join(threads[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed = (end.tv_sec - start.tv_sec)
        + (end.tv_nsec - start.tv_nsec) / 1e9;
    return elapsed;
```

```
}

int main() {
    int thread_counts[] = {1, 2, 4, 8};
    for (int t = 0; t < 4; t++) {
        int n = thread_counts[t];
        list_t *lc = list_create();
        list_t *lf = list_create();
        double tc = run_benchmark(lc,
            coarse_insert, coarse_contains,
            coarse_delete, n);
        double tf = run_benchmark(lf,
            fine_insert, fine_contains,
            fine_delete, n);
        int ops = n * OPS_PER_THREAD;
        printf("%d threads: coarse=%.4fs (%.0f ops/s) fine=%.4fs (%.0f
ops/s)\n",
            n, tc, ops/tc, tf, ops/tf);
    }
    return 0;
}
```

Questions

Q1.1: Present the throughput table from the benchmark output. Which version scales better with more threads and why?

Q1.2: In the fine-grained version, why is it essential to use hand-over-hand locking (never releasing the previous lock before acquiring the next)? What can go wrong if you release the previous lock first?

Exercise 2: Lock-Free Counter with CAS

Background

As discussed in the lecture, a shared counter protected by a lock serializes all increments. Using CAS, we can build a lock-free counter. Using atomic fetch-and-add, we get the simplest and fastest solution.

Provided Code

The following program includes the unsafe and mutex versions plus the complete benchmark. You only need to implement the functions marked TODO.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define NUM_INCREMENTS 1000000

long counter = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

// Version 1: No synchronization (broken)
```

```
void *inc_unsafe(void *arg) {
    for (int i = 0; i < NUM_INCREMENTS; i++)
        counter++;
    return NULL;
}

// Version 2: Mutex (correct, blocking)
void *inc_mutex(void *arg) {
    for (int i = 0; i < NUM_INCREMENTS; i++) {
        pthread_mutex_lock(&mutex);
        counter++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

// ========================================
// TODO: Implement these two functions
// ========================================

// Version 3: CAS loop (lock-free)
// Read counter, attempt CAS(counter, old, old+1)
// Retry on failure
void *inc_cas(void *arg) {
    // TODO
    return NULL;
}

// Version 4: Atomic fetch-and-add (wait-free)
// Use __sync_fetch_and_add(&counter, 1)
void *inc_atomic(void *arg) {
    // TODO
    return NULL;
}

// --- Benchmark harness (do not modify) ---
typedef struct {
    const char *name;
    void *(*func)(void *);
} version_t;

int main() {
    version_t versions[] = {
        {"Unsafe ", inc_unsafe},
        {"Mutex  ", inc_mutex},
        {"CAS    ", inc_cas},
        {"Atomic ", inc_atomic},
    };
    int thread_counts[] = {1, 2, 4, 8};

    printf("%-8s %3s %12s %10s %8s\n",
           "Version","Thr","Counter","Expected",
```

```
            "Time(s)");
    printf("-----------------------------------------------\n");

    for (int v = 0; v < 4; v++) {
        for (int t = 0; t < 4; t++) {
            int n = thread_counts[t];
            long expected = (long)n * NUM_INCREMENTS;
            counter = 0;
            pthread_t threads[n];
            struct timespec start, end;

            clock_gettime(CLOCK_MONOTONIC, &start);
            for (int i = 0; i < n; i++)
                pthread_create(&threads[i], NULL,
                    versions[v].func, NULL);
            for (int i = 0; i < n; i++)
                pthread_join(threads[i], NULL);
            clock_gettime(CLOCK_MONOTONIC, &end);

            double elapsed = (end.tv_sec-start.tv_sec)
                + (end.tv_nsec-start.tv_nsec)/1e9;
            printf("%-8s %3d %12ld %10ld %8.4f %s\n",
                versions[v].name, n, counter,
                expected, elapsed,
                counter==expected ? "OK" : "WRONG");
        }
        printf("\n");
    }
    return 0;
}
```

Questions

Q2.1: Copy the benchmark output table into your report. Which versions produce correct results? Which is fastest with 8 threads?

Q2.2: Explain why the CAS version (Version 3) is lock-free but NOT wait-free. Could a single thread theoretically starve under extreme contention?

Q2.3: Why is Version 4 (fetch-and-add) faster than Version 3 (CAS loop) under high contention?

Exercise 3: Bounded Queue with Split Locks

Background

As covered in the lecture, a bounded queue can use two separate locks — one for enqueue and one for dequeue — allowing concurrent enqueues and dequeues. An atomic permits counter tracks capacity.

Provided Code

The struct, initialization, and the complete test harness are provided. You only need to implement queue_enq() and queue_deq().

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef struct node {
    int value;
    struct node *next;
} node_t;

typedef struct {
    node_t *head;
    node_t *tail;
    pthread_mutex_t enqLock;
    pthread_mutex_t deqLock;
    pthread_cond_t notFull;
    pthread_cond_t notEmpty;
    volatile int permits;
    int capacity;
} bounded_queue_t;

void queue_init(bounded_queue_t *q, int cap) {
    node_t *sentinel = malloc(sizeof(node_t));
    sentinel->value = -1;
    sentinel->next = NULL;
    q->head = sentinel;
    q->tail = sentinel;
    q->capacity = cap;
    q->permits = cap;
    pthread_mutex_init(&q->enqLock, NULL);
    pthread_mutex_init(&q->deqLock, NULL);
    pthread_cond_init(&q->notFull, NULL);
    pthread_cond_init(&q->notEmpty, NULL);
}

// ========================================
// TODO: Implement these two functions
// ========================================

// Enqueue: acquire enqLock, wait if permits==0,
// add node at tail, decrement permits atomically,
// release enqLock, signal notEmpty if needed.
void queue_enq(bounded_queue_t *q, int value) {
    // TODO
}

// Dequeue: acquire deqLock, wait if head==tail
// (empty), remove node after sentinel,
// increment permits atomically, release deqLock,
// signal notFull if needed. Return the value.
int queue_deq(bounded_queue_t *q) {
    // TODO
    return -1;
```

```
}
```

Test Harness (provided)

```
#define ITEMS_PER_PRODUCER 10000

typedef struct {
    bounded_queue_t *q;
    int id;
} targ_t;

void *producer(void *arg) {
    targ_t *ta = (targ_t *)arg;
    for (int i = 0; i < ITEMS_PER_PRODUCER; i++)
        queue_enq(ta->q, ta->id * 100000 + i);
    return NULL;
}

long consumer_sum = 0;
int  consumer_count = 0;
pthread_mutex_t sum_lock = PTHREAD_MUTEX_INITIALIZER;

void *consumer(void *arg) {
    targ_t *ta = (targ_t *)arg;
    while (1) {
        int val = queue_deq(ta->q);
        if (val == -999) break;  // poison pill
        pthread_mutex_lock(&sum_lock);
        consumer_sum += val;
        consumer_count++;
        pthread_mutex_unlock(&sum_lock);
    }
    return NULL;
}

void run_test(int np, int nc, int cap) {
    bounded_queue_t q;
    queue_init(&q, cap);
    consumer_sum = 0;
    consumer_count = 0;

    pthread_t prods[np], cons[nc];
    targ_t pargs[np], cargs[nc];
    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < nc; i++) {
        cargs[i] = (targ_t){&q, i};
        pthread_create(&cons[i], NULL,
                       consumer, &cargs[i]);
    }
    for (int i = 0; i < np; i++) {
        pargs[i] = (targ_t){&q, i};
```

```
        pthread_create(&prods[i], NULL,
                       producer, &pargs[i]);
    }
    for (int i = 0; i < np; i++)
        pthread_join(prods[i], NULL);
    // Send poison pills to stop consumers
    for (int i = 0; i < nc; i++)
        queue_enq(&q, -999);
    for (int i = 0; i < nc; i++)
        pthread_join(cons[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed = (end.tv_sec - start.tv_sec)
        + (end.tv_nsec - start.tv_nsec) / 1e9;
    int expected = np * ITEMS_PER_PRODUCER;
    printf("%dP/%dC cap=%d: %.4fs, items=%d/%d %s\n",
        np, nc, cap, elapsed, consumer_count,
        expected,
        consumer_count == expected ? "OK" : "WRONG");
}

int main() {
    printf("--- Split-Lock Queue Tests ---\n");
    run_test(1, 1, 10);
    run_test(2, 2, 10);
    run_test(4, 4, 10);
    run_test(4, 4, 100);
    return 0;
}
```

Questions

Q3.1: Present the test output. Does item count match expected for all configurations? How does throughput change with more producers/consumers?

Q3.2: From the lecture, explain how the split counter optimization (enqSidePermits and deqSidePermits) reduces synchronization overhead. Why is accessing a shared permits counter a bottleneck? (Implementation not required.)

Q3.3: Why must the queue use pthread_cond_broadcast() rather than pthread_cond_signal() when notifying? Describe the lost-wakeup scenario from the lecture.

Exercise 4: Progress Conditions Analysis

Background

From the lecture, recall the progress condition hierarchy:

- Wait-free: Every thread completes in a finite number of its own steps.
- Lock-free: At least one thread makes progress in a finite number of steps (but individual threads might starve).
- Obstruction-free: A thread makes progress only if it runs in isolation.
- Blocking: A delayed thread can prevent others from making progress.

Task

Classify each of the following as wait-free, lock-free, obstruction-free, or blocking. Justify each answer in 1–2 sentences.

Snippet A:

```
void increment() {
    pthread_mutex_lock(&m);
    counter++;
    pthread_mutex_unlock(&m);
}
```

Snippet B:

```
void increment() {
    long old;
    do {
        old = counter;
    } while (!__sync_bool_compare_and_swap(
                &counter, old, old + 1));
}
```

Snippet C:

```
void increment() {
    __sync_fetch_and_add(&counter, 1);
}
```

Snippet D:

```
void increment() {
    long old = counter;
    if (!__sync_bool_compare_and_swap(
            &counter, old, old + 1))
        return;  // give up on failure
}
```

Questions

Q4.1: Classify each snippet (A, B, C, D) and provide a brief justification for each.

Q4.2: Which progress condition would you choose for a real-time system where every operation must complete within bounded time? Why?

Submission Guidelines

- Submit source files (.c), one per exercise, named as: Lab05_<IndexNumber1>_(IndexNumber2>_Ex<N>.c
- For Exercise 4, include your answers in the PDF report (no code required).
- Submit one PDF report with benchmark output, throughput analysis, and answers to all questions.
- Compile Exercises 1–3 with: gcc -pthread -O2 -o output source.c
- Submit the final zip file via Moodle before the deadline.
